

Лабораторная работа 2. LLM и Трансформер

Список задач:

1. Введение и подготовка данных:

- Подготовить текстовый корпус для обучения модели.
- Разделить данные на тренировочный, валидационный и тестовый наборы.
- Очистить данные с применением методов, таких как удаление пунктуации, стоп-слов, и лемматизация.
- Преобразовать текст в числовое представление через токенизацию.

2. Изучение архитектуры трансформера:

- Разобраться в основных компонентах трансформера:
 - Self-Attention.
 - Multi-Head Attention.
 - Feed-Forward Network.
- Изучить механизмы позиционного кодирования и остаточных связей.
- Описать ключевые преимущества трансформера по сравнению с RNN и LSTM.

3. Создание модели на основе трансформера:

- Инициализировать трансформер (например, GPT, BERT) и подготовить его для fine-tuning.
- Настроить гиперпараметры модели, такие как количество слоев, размерность векторного представления, число эпох и скорость обучения.

4. Обучение модели:

- Провести обучение трансформера на подготовленных данных.
- Применить различные оптимизаторы (Adam, SGD, RMSProp) и исследовать их влияние на процесс обучения.
- Использовать learning rate scheduler для управления скоростью обучения.

5. Оценка модели:

- Измерить производительность модели с использованием метрик:
 - **Perplexity** для оценки уверенности модели.
 - **BLEU** и **ROUGE** для оценки качества генерации текста.
- Сравнить результаты при использовании разных оптимизаторов и scheduler'ов.

6. Интерактивное тестирование модели:

- Разработать интерфейс для взаимодействия с моделью.
- Реализовать чат-бота, способного отвечать на запросы пользователей.
- Настроить параметры генерации (например, max_length, top_k, top_p) и проанализировать их влияние на ответы модели.

7. Анализ результатов:

- Провести эксперименты с различными гиперпараметрами и методами обучения.
- Построить графики зависимости потерь и Perplexity от числа эпох.

- Провести ручную оценку сгенерированных ответов на смысловую релевантность.

8. Дополнительные задачи (опционально):

- Добавить логирование запросов и ответов в текстовый файл.
- Реализовать графический интерфейс для чат-бота.
- Провести эксперименты с мультимодальными трансформерами (например, обработка текста и изображений).

Что включить в отчет:

1. Теоретическое описание методов и архитектуры трансформера.
2. Скриншоты сгенерированных текстов, графиков и метрик.
3. Таблицы с результатами тестирования различных гиперпараметров.
4. Выводы по каждому этапу работы

Основы и развитие больших языковых моделей (LLM)

1. Введение в большие языковые модели (LLM)

Большие языковые модели (LLM) — это модели, созданные для обработки естественного языка. Они обучены на больших объемах текстовых данных и способны выполнять задачи понимания и генерации текста. Основная цель LLM — прогнозировать следующее слово в последовательности слов, на основе которого они могут решать различные задачи, включая перевод, суммаризацию, генерацию текста и даже программирование.

Пример реального применения: Модели, такие как GPT-4 и BERT, используются в чат-ботах, виртуальных помощниках (например, Siri, Google Assistant), автоматической модерации контента, переводе и в других областях, где требуется обработка естественного языка.

2. Архитектура трансформера и его роль в LLM

Современные LLM базируются на архитектуре трансформеров, представленной в статье "*Attention is All You Need*" (2017) командой Google. Трансформеры обеспечивают высокую производительность благодаря параллельной обработке данных и способности учитывать контекст на уровне всего текста.

1. Проблемы предыдущих архитектур

До появления трансформеров задачи машинного перевода и другие задачи обработки последовательностей текста часто решались с помощью рекуррентных нейронных сетей (например, LSTM или GRU). Однако такие сети имели несколько проблем.

- Медленное обучение. Рекуррентные сети обрабатывают данные последовательно, что затрудняет параллелизацию.
- Проблемы долгосрочной памяти. Сложно обрабатывать длинные последовательности, так как информация со временем "затухает".

2. Трансформер

Трансформер был предложен как архитектура, которая устраняет зависимость от последовательного ввода данных и использует механизм внимания (attention) для обработки текстовых последовательностей целиком. Это позволяет эффективнее обрабатывать длинные контексты и значительно ускоряет процесс обучения.

Архитектура трансформера

Архитектура трансформера состоит из двух основных блоков:

1. Encoder (кодировщик) — преобразует входную последовательность в более абстрактное представление.
2. Decoder (декодировщик) — преобразует представление, созданное кодировщиком, в выходную последовательность.

Encoder и его компоненты

Каждый блок кодировщика включает два подблока:

1. Многоголовое внимание (Multi-Head Self-Attention): Самовнимание позволяет каждому слову учитывать важность всех остальных слов в предложении, а многоголовое внимание позволяет сети захватывать различные типы зависимостей между словами.
2. Feed-Forward Network: Полносвязная нейронная сеть, которая обрабатывает каждое слово независимо, применяя нелинейные преобразования.

Каждый блок также использует слой нормализации и добавляет остаточные связи (residual connections), которые помогают сохранить информацию от предыдущих шагов и ускоряют обучение.

Decoder и его компоненты

Декодировщик состоит из трех подблоков:

1. Masked Multi-Head Attention: Механизм внимания, скрывающий будущие слова, чтобы модель не смотрела вперед при генерации последовательности.
2. Multi-Head Attention: Этот подблок учитывает выходы кодировщика и связывает их с текущей последовательностью декодировщика.
3. Feed-Forward Network: Как и в кодировщике, полносвязная сеть обрабатывает каждое слово отдельно.

Механизм внимания (Attention Mechanism)

Механизм внимания является центральной концепцией архитектуры трансформера. Его основная задача — присваивать разный "вес" словам в последовательности, определяя их значимость относительно текущего слова.

Основная формула self-attention

Механизм внимания основан на трех матрицах: Query (запрос), Key (ключ) и Value (значение). Эти матрицы используются для вычисления значимости каждого слова относительно других в предложении.

1. Вычисление значимости:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

где Q , K , и V — это матрицы запросов, ключей и значений, которые определяются для каждого слова в последовательности, а d_k — размерность ключей.

2. Softmax: Нормализует значения, чтобы они суммировались до 1, и придает каждому слову "вес" или важность в зависимости от его контекста.
3. Многоголовое внимание (Multi-Head Attention): Вместо одного внимания трансформер использует несколько голов, каждая из которых обрабатывает разные части информации. Это позволяет модели захватывать различные типы отношений между словами.

Пример работы внимания:

В предложении "Кошка сидит на ковре, а собака спит рядом" механизм внимания будет обращать больше внимания на слова "собака" и "спит" при обработке слова "спит", а слова "ковер" и "кошка" будут иметь меньший вес для текущего контекста.

Позиционное кодирование (Positional Encoding)

Поскольку трансформеры обрабатывают слова параллельно, им необходимо дополнительное представление для учета порядка слов в последовательности. В трансформере для этого используется позиционное кодирование — значения, добавляемые к векторным представлениям слов, чтобы учитывать их позицию в предложении.

Формула позиционного кодирования:

$$PE(pos, 2i) = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$PE(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

где pos — позиция слова, а d_{model} — размерность векторного представления. Это позволяет модели понимать, что порядок слов в предложении важен.

Преимущества и результаты применения трансформеров

Трансформеры продемонстрировали значительное улучшение по сравнению с предыдущими моделями в задачах машинного перевода, суммаризации текста, генерации речи и других задачах. Среди преимуществ:

1. Параллельная обработка: Трансформеры позволяют обрабатывать данные параллельно, что делает обучение и генерацию намного быстрее по сравнению с RNN и LSTM.
2. Улучшенное захватывание контекста: Благодаря механизму self-attention трансформеры могут учитывать важные слова в предложении независимо от их расположения.
3. Масштабируемость: Благодаря возможности параллелизации трансформеры легко масштабируются и могут обучаться на больших объемах данных.

Примеры применения трансформеров:

- Google Translate: В 2018 году Google внедрил трансформеры в переводчик, что улучшило качество перевода.
- BERT и GPT: Модели BERT и GPT, основанные на архитектуре трансформера, привели к новым достижениям в задачах NLP, включая классификацию текста, суммаризацию и ответ на вопросы.

Тенденции и развитие трансформеров

С момента публикации статьи *"Attention is All You Need"* развитие трансформеров стремительно продолжается:

1. BERT (Bidirectional Encoder Representations from Transformers): Эта модель использует двунаправленный контекст, чтобы лучше понимать смысл предложения, улучшая задачи понимания текста.
2. GPT (Generative Pre-trained Transformer): В отличие от BERT, GPT фокусируется на генерации текста, обучаясь предсказывать следующее слово. Модели GPT, такие как GPT-3 и GPT-4, могут решать широкий спектр задач, включая написание кода и перевод текста.
3. T5 и BART: Модели T5 и BART применяются для задач, где требуется как генерация текста, так и понимание, например, для суммаризации и перевода.

3. Масштабирование LLM: Тренды и технологии

С увеличением объема данных и вычислительных мощностей стало возможным создание моделей с миллиардами и даже триллионами параметров. Например, модели GPT-3 (175 миллиардов параметров) и GPT-4 (считается, что более 1 триллиона параметров) демонстрируют значительно улучшенные результаты, особенно в сложных задачах.

Тенденции в масштабировании:

1. Увеличение параметров: Чем больше параметров у модели, тем лучше она способна моделировать сложные зависимости в языке.
2. Мультимодальные модели: Современные LLM могут работать не только с текстом, но и с другими типами данных, такими как изображения (DALL-E от OpenAI).
3. Специализированные архитектуры и адаптивные модели: Вместо того, чтобы увеличивать размер модели, некоторые исследователи работают над адаптивными архитектурами, которые эффективно используют ресурсы.

Пример масштабирования:

Компания OpenAI и Google инвестируют значительные ресурсы в исследования масштабирования моделей, чтобы сделать их более производительными и точными. В реальном мире это приводит к созданию ассистентов, таких как ChatGPT и Bard, которые могут проводить сложные диалоги, помогать в написании кода и автоматизировать множество задач.

4. Методы обучения и оптимизации LLM

Модель должна быть обучена на большом количестве текстов, что требует использования специальных методов оптимизации и управления скоростью обучения.

- Оптимизаторы: Adam, RMSProp и их вариации обычно применяются для обучения LLM. Эти оптимизаторы обеспечивают быстрое схождение и стабильное обучение.
- Scheduler'ы для скорости обучения: Использование таких методов, как линейное затухание (linear decay) и warm-up, помогает модели достигать оптимальных параметров.
Например, `get_linear_schedule_with_warmup` плавно уменьшает скорость обучения, что позволяет модели избежать резких изменений.

Реальный пример использования:

В компании Microsoft применяют методы оптимизации и scheduler'ы для своих моделей, таких как Azure Cognitive Services. Благодаря этому, модели могут обрабатывать запросы пользователей более точно и быстро.

5. Оценка моделей с помощью метрик

Для оценки качества генерации текста в LLM используют метрики BLEU, ROUGE и Perplexity:

- BLEU (Bilingual Evaluation Understudy) — используется для оценки качества перевода.
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation) — подходит для задач суммаризации.
- Perplexity — используется для оценки языковых моделей и показывает степень уверенности модели.

Пример применения метрик:

В Google Translate и Amazon Alexa применяют BLEU и ROUGE для оценки качества переводов и ответов на вопросы. Это позволяет следить за улучшением модели и быстро обнаруживать ошибки.

6. Примеры реального применения LLM

LLM уже используются во многих сферах, и их влияние продолжает расти:

- Чат-боты и виртуальные помощники: LLM применяются для создания чат-ботов, способных отвечать на вопросы пользователей и помогать в решении задач. Пример: ChatGPT от OpenAI.
- Автоматическая модерация контента: Модели помогают анализировать пользовательский контент, определять токсичные комментарии и предотвращать распространение дезинформации. Пример: автоматическая модерация комментариев в YouTube.
- Перевод и обработка текста: LLM позволяют автоматизировать перевод и суммаризацию текста, что упрощает работу в бизнесе и науке. Пример: Google Translate.
- Создание кода: Модели, такие как GitHub Copilot, обученные на коде, помогают программистам ускорить написание кода, предоставляя автоматические подсказки.
- Мультимодальные модели: LLM, такие как DALL-E и CLIP, могут работать не только с текстом, но и с изображениями. Они позволяют создавать изображения по текстовым описаниям, что открывает новые возможности для креативных индустрий.

7. Этика и ограничения LLM

Несмотря на многочисленные достижения, LLM сталкиваются с рядом этических и технологических ограничений:

- **Этика:** Существует опасность использования LLM для создания фейковых новостей, дезинформации и токсичного контента. Компании внедряют методы фильтрации и контроля для борьбы с этими проблемами.
- **Ограничения:** LLM требуют значительных вычислительных ресурсов, что делает их недоступными для малых компаний. Кроме того, большие языковые модели не всегда точны и могут генерировать несоответствующий контент.

Введение: Список задач

Список задач

1. **Загрузка и подготовка корпуса данных:** Загрузить текстовый корпус, оценить его объем и структуру.
2. **Очистка данных:** Применить разные методы очистки данных, такие как удаление пунктуации, стоп-слов, лемматизация, и оценить их влияние.
3. **Токенизация данных:** Преобразовать текстовые данные в числовые представления для последующей обработки трансформером.
4. **Создание и настройка модели:** Инициализировать модель трансформера и настроить гиперпараметры для обучения.
5. **Обучение модели:** Провести обучение с использованием разных оптимизаторов и сравнить результаты.
6. **Оптимизация модели:** Применить различные методы управления скоростью обучения и оценить их влияние на модель.
7. **Оценка модели с использованием метрик BLEU, ROUGE и Perplexity:** Измерить качество модели с использованием метрик и сравнить результаты.
8. **Интерактивное тестирование модели:** Реализовать интерфейс для взаимодействия с моделью и протестировать ответы модели на различные вопросы.

Определения терминов

1. Трансформер (Transformer)

Архитектура нейронной сети, которая использует self-attention для обработки последовательностей текста.

```
from transformers import TransformerModel
```

Инициализация простой модели трансформера

```
model = TransformerModel(num_layers=6, d_model=512, num_heads=8)
```

2. Механизм внимания (Attention Mechanism)

Позволяет модели фокусироваться на разных частях последовательности.

```

import torch
import torch.nn.functional as F

# Пример механизма внимания
Q = torch.rand(3, 5) # Запросы
K = torch.rand(3, 5) # Ключи
V = torch.rand(3, 5) # Значения

attention_scores = torch.matmul(Q, K.T)
attention_weights = F.softmax(attention_scores, dim=-1)
attention_output = torch.matmul(attention_weights, V)

```

3. Self-Attention (Самовнимание)

Обрабатывает каждый токен, оценивая его значимость относительно всех остальных токенов в предложении.

```

import numpy as np

# Входные данные
Q = np.random.rand(4, 64) # Запросы
K = np.random.rand(4, 64) # Ключи
V = np.random.rand(4, 64) # Значения
d_k = K.shape[1]

# Вычисление self-attention
attention_weights = np.dot(Q, K.T) / np.sqrt(d_k)
attention_output = np.dot(attention_weights, V)

```

4. Multi-Head Attention (Многоголовое внимание)

Использует несколько "голов" внимания для захвата различных аспектов контекста.

```

from transformers import MultiHeadAttention

# Пример использования многоголового внимания
attention_layer = MultiHeadAttention(embed_dim=64,
num_heads=8)

```

5. Feed-Forward Network (Полносвязная нейронная сеть)

Линейный слой, который обрабатывает выходы self-attention.

```

import torch.nn as nn

# Пример полносвязной сети
feed_forward = nn.Sequential(
    nn.Linear(512, 2048),
    nn.ReLU(),
    nn.Linear(2048, 512)
)

```

6. Encoder (Кодировщик)

Блок, который преобразует входную последовательность в абстрактное представление.

```
from transformers import Encoder

# Инициализация кодировщика
encoder = Encoder(num_layers=6, d_model=512, num_heads=8)
```

7. Decoder (Декодировщик)

Использует выходы кодировщика для генерации выходной последовательности.

```
from transformers import Decoder

# Инициализация декодировщика
decoder = Decoder(num_layers=6, d_model=512, num_heads=8)
```

8. Позиционное кодирование (Positional Encoding)

Добавляет информацию о положении токенов.

```
import numpy as np

def positional_encoding(position, d_model):
    angle_rates = 1 / np.power(10000, (2 *
(np.arange(d_model) // 2)) / np.float32(d_model)))
    angle_rads = position * angle_rates
    angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
    angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])
    return angle_rads

position = 10
d_model = 512
pos_encoding = positional_encoding(position, d_model)
```

9. BLEU

Метрика для оценки качества перевода.

```
from nltk.translate.bleu_score import sentence_bleu

reference = [['это', 'пример', 'предложения']]
candidate = ['это', 'пример']
bleu_score = sentence_bleu(reference, candidate)
print("BLEU Score:", bleu_score)
```

10. ROUGE

Метрика для оценки суммаризации текста.

```
from rouge_score import rouge_scorer
```

```

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2',
'rougeL'], use_stemmer=True)
scores = scorer.score('это сгенерированный текст', 'это
эталонный текст')
print("ROUGE Score:", scores)

```

11. Perplexity

Показывает, насколько модель уверена в своих предсказаниях.

```

import math

def calculate_perplexity(loss):
    return math.exp(loss)

# Пример вычисления перплексии
loss = 2.5
print("Perplexity:", calculate_perplexity(loss))

```

12. LLM (Large Language Model)

Большая языковая модель, способная решать широкий спектр задач.

```

from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Загрузка предобученной LLM
model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

```

13. BERT

Модель для обработки текста, использующая двунаправленное внимание.

```

from transformers import BertTokenizer, BertModel

# Загрузка предобученной модели BERT
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

```

14. GPT

Модель, ориентированная на генерацию текста.

```

from transformers import GPT2Tokenizer, GPT2LMHeadModel

# Загрузка предобученной модели GPT-2
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

```

15. Masked Attention

Ограничивает внимание только на предыдущие токены.

```

from transformers import GPT2LMHeadModel

```

```
# Пример маскированного внимания в GPT-2
model = GPT2LMHeadModel.from_pretrained('gpt2')
```

16. Optimization (Оптимизация)

Процесс минимизации функции потерь.

```
import torch.optim as optim

# Пример оптимизатора Adam
optimizer = optim.Adam(model.parameters(), lr=5e-5)
```

17. Learning Rate Scheduler

Метод управления скоростью обучения.

```
from transformers import get_linear_schedule_with_warmup

# Пример scheduler'a
scheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=100, num_training_steps=1000)
```

18. Warmup

Постепенное увеличение скорости обучения.

```
from transformers import get_linear_schedule_with_warmup

# Пример warmup scheduler'a
scheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=100, num_training_steps=1000)
```

19. N-grams

Последовательности из nn слов для анализа текста.

```
from nltk import ngrams

sentence = "This is an example sentence"
n_grams = list(ngrams(sentence.split(), 2))
print("2-grams:", n_grams)
```

20. Fine-tuning

Настройка предобученной модели на конкретной задаче.

```
from transformers import AdamW

# Пример настройки оптимизатора для fine-tuning
optimizer = AdamW(model.parameters(), lr=2e-5)
```

21. Tokenization

Преобразование текста в токены.

```

from transformers import GPT2Tokenizer

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
tokens = tokenizer.tokenize("Hello, world!")
print("Tokens:", tokens)

```

22. Pre-training

Предобучение модели на большом объеме данных.

```

from transformers import BertModel

# Пример загрузки предобученной модели
model = BertModel.from_pretrained('bert-base-uncased')

```

23. Truncation

Обрезка текста до заданной длины.

```

# Пример обрезки токенов
tokens = tokenizer.encode("This is an example sentence.",
max_length=5, truncation=True)
print("Truncated tokens:", tokens)

```

24. Zero-shot и Few-shot обучение

Методы для выполнения задачи с малым количеством примеров или без них.

```

from transformers import pipeline

# Пример zero-shot классификации
classifier = pipeline("zero-shot-classification")
result = classifier("This is a sample sentence.",
candidate_labels=["positive", "negative"])
print("Zero-shot classification result:", result)

```

25. Encoder-Decoder Architecture

Архитектура, состоящая из кодировщика и декодировщика.

```

from transformers import EncoderDecoderModel

# Загрузка предобученной модели encoder-decoder
model =
EncoderDecoderModel.from_encoder_decoder_pretrained("bert
-base-uncased", "gpt2")

```

1. Введение и подготовка данных

Задание: Подготовить текстовый корпус для обучения модели, разбить его на тренировочный, валидационный и тестовый наборы данных.

Шаги решения:

1. Поиск корпуса текстов:

- Используйте открытые датасеты с диалогами, такие как *Cornell Movie Dialogues* (<https://www.kaggle.com/datasets/rajathmc/cornell-moviedialog-corpus/data>), *Reddit Conversational Data* (<https://www.kaggle.com/datasets/jerryqu/reddit-conversations/data>), или *OpenSubtitles*.
- Загрузите данные и оцените их объем и структуру.

Для загрузки данных с диалогами можно использовать уже известные наборы данных. Например, данные *Cornell Movie Dialogues* или *Reddit Conversational Data*. Здесь показан пример с использованием датасета `cornell_movie_dialogues.txt`.

```
# Пример чтения данных из текстового файла (Cornell Movie Dialogues)
def load_data(file_path):
    with open(file_path, 'r', encoding='utf-8') as file:
        data = file.readlines()
    return data

# Загрузка данных и вывод информации о наборе данных
file_path = 'cornell_movie_dialogues.txt' # Замените на
путь к вашему файлу
data = load_data(file_path)
print("Количество строк:", len(data))
print("Пример строки:", data[0]) # Вывод первой строки
для ознакомления
```

Этот код загружает данные и выводит общее количество строк, а также пример строки для анализа структуры данных.

2. Очистка данных:

Почему важна очистка данных?

1. **Снижение шума:** Шум (лишние данные) может мешать модели находить закономерности, необходимые для точного предсказания.
2. **Сокращение словаря:** Удаление пунктуации, стоп-слов и приведение к нижнему регистру уменьшает размер словаря, что снижает сложность модели.
3. **Улучшение производительности модели:** Применение стемминга и лемматизации приводит к уменьшению объема данных и ускоряет процесс обучения, сохраняя при этом смысловую нагрузку.
4. **Унификация:** Единообразие в данных позволяет модели лучше понимать и обрабатывать текст, что особенно важно для задач классификации и генерации текста.

1. Преобразование текста в нижний регистр

Приведение всех символов к нижнему регистру помогает унифицировать данные и устранить различия между одинаковыми словами, записанными в разных регистрах. Например, слова "apple" и "Apple" будут трактоваться как одинаковые.

- **Цель:** уменьшить разнообразие токенов и снизить размер словаря.
- **Недостаток:** иногда регистр может нести смысловую нагрузку, например, имена собственные или начало предложения.

2. Удаление HTML-тегов и спецсимволов

При обработке текста из интернета часто встречаются HTML-теги (<p>, <a>, <div>) и специальные символы (, &). Их удаление делает текст чище и упрощает его обработку.

- **Цель:** удалить визуальные или служебные символы, которые не несут смысловой нагрузки для модели.
- **Метод:** Использование регулярных выражений (`re.sub(r'<.*?>', '', text)`) для удаления HTML-тегов и спецсимволов.

3. Удаление ссылок и упоминаний

Часто в текстах встречаются ссылки на веб-страницы или упоминания других пользователей (например, @username в социальных сетях). Обычно эти элементы не несут значимой информации для анализа и могут быть удалены.

- **Цель:** уменьшить шум в данных, особенно в текстах из социальных сетей.
- **Метод:** Регулярные выражения для поиска и удаления ссылок (`re.sub(r'http\S+|www\S+', '', text)`) и упоминаний (`re.sub(r'@\w+', '', text)`).

4. Удаление знаков пунктуации

Пунктуация может быть полезна для анализа, но в большинстве случаев она добавляет лишние токены, увеличивая размер словаря и усложняя обучение модели. Удаление пунктуации делает данные более унифицированными.

- **Цель:** сократить количество уникальных токенов и сделать текст более однородным.
- **Метод:** Использование регулярных выражений, например, `re.sub(r'[^a-zA-Z\s]', '', text)`, чтобы оставить только буквы и пробелы.

5. Удаление стоп-слов

Стоп-слова — это часто встречающиеся слова (например, «и», «в», «на»), которые не несут значимой информации для анализа текста. Удаление стоп-слов позволяет сосредоточиться на содержательных словах, которые несут смысл.

- **Цель:** уменьшить количество слов в тексте, оставляя только значимые термины.
- **Недостаток:** в зависимости от задачи некоторые стоп-слова могут нести смысловую нагрузку и могут быть сохранены.
- **Метод:** Библиотеки NLP, такие как NLTK и spaCy, предоставляют списки стоп-слов для различных языков.

6. Лемматизация и стемминг

Эти методы помогают привести слова к их базовой форме, что позволяет уменьшить количество токенов, сохраняя смысл. Лемматизация переводит слово в его словарную форму (например, «cats» → «cat»), а стемминг удаляет окончания для получения основы слова (например, «running» → «run»).

- **Цель:** уменьшить количество уникальных токенов, сохраняя основное значение слов.
- **Недостаток:** стемминг может привести к неточным основам, так как метод просто отрезает окончания.
- **Метод:** использование библиотек (например, NLTK для стемминга и лемматизации, или spaCy для лемматизации).

- Преобразуйте весь текст в нижний регистр для единообразия.
- Удалите ненужные символы и знаки пунктуации, которые не несут полезной информации (например, HTML-теги, спецсимволы).
- Используйте библиотеку, например, NLTK или SpaCy, для токенизации текста (разделения на слова и предложения).

3. Примеры и пояснения

4. Пример 1. Преобразование текста в нижний регистр и удаление пунктуации

```
import re

text = "Hello, World! Welcome to AI."
text = text.lower() # Преобразование в нижний регистр
text = re.sub(r'[\w\s]', '', text) # Удаление знаков
пунктуации
print(text) # Вывод: "hello world welcome to ai"
```

5. Пример 2. Удаление стоп-слов

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
nltk.download('stopwords')
nltk.download('punkt')

text = "This is a simple example showing how to remove
stop words."
stop_words = set(stopwords.words('english'))
words = word_tokenize(text)
filtered_text = [word for word in words if word.lower()
not in stop_words]
print(filtered_text) # Вывод: ['simple', 'example',
'showing', 'remove', 'stop', 'words']
```

6. Пример 3. Лемматизация

```
from nltk.stem import WordNetLemmatizer
import nltk
nltk.download('wordnet')
nltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()
text = "The cats are running faster than the other
animals."
words = word_tokenize(text)
lemmatized_words = [lemmatizer.lemmatize(word) for word
in words]
```

```
print(lemmatized_words) # Вывод: ['The', 'cat', 'are',  
'running', 'faster', 'than', 'the', 'other', 'animal']
```

Выполнение

Для очистки данных воспользуемся библиотеками `re` (регулярные выражения) и `nltk` или `sraCy` для токенизации.

```
import re  
import nltk  
from nltk.tokenize import word_tokenize  
nltk.download('punkt') # Загрузка токенизатора  
  
# Функция очистки текста  
def clean_text(text):  
    # Преобразование текста в нижний регистр  
    text = text.lower()  
    # Удаление HTML-тегов, ссылок и спецсимволов  
    text = re.sub(r'<.*?>', '', text) # Удаление  
HTML  
    text = re.sub(r'http\S+|www\S+', '', text) # Удаление  
ссылок  
    text = re.sub(r'^a-zA-Z\s]', '', text) # Удаление  
спецсимволов и знаков пунктуации  
    return text  
  
# Токенизация текста  
def tokenize_text(text):  
    # Используем word_tokenize из NLTK для разделения  
текста на слова  
    tokens = word_tokenize(text)  
    return tokens  
  
# Пример использования  
sample_text = data[0]  
cleaned_text = clean_text(sample_text)  
tokens = tokenize_text(cleaned_text)  
print("Оригинальный текст:", sample_text)  
print("Очищенный текст:", cleaned_text)  
print("Токены:", tokens)
```

Здесь `clean_text` очищает текст от ненужных символов, а `tokenize_text` разбивает очищенный текст на токены.

7. Разделение данных:

- Разделите данные на тренировочный (70%), валидационный (15%) и тестовый (15%) наборы.
- Обеспечьте случайное распределение данных, чтобы все части были репрезентативными.

Для разделения данных мы используем функцию `train_test_split` из библиотеки `scikit-learn`.


```

from sklearn.model_selection import train_test_split

# Разделение данных на тренировочный, валидационный и
тестовый наборы
def split_data(data, train_ratio=0.7, val_ratio=0.15,
test_ratio=0.15):
    # Сначала разделяем данные на тренировочный и
временный набор
    train_data, temp_data = train_test_split(data,
test_size=(1 - train_ratio), random_state=42)
    # Разделяем временный набор на валидационный и
тестовый наборы
    val_data, test_data = train_test_split(temp_data,
test_size=test_ratio / (test_ratio + val_ratio),
random_state=42)
    return train_data, val_data, test_data

# Пример использования
train_data, val_data, test_data = split_data(data)
print("Размер тренировочного набора:", len(train_data))
print("Размер валидационного набора:", len(val_data))
print("Размер тестового набора:", len(test_data))

```

8. Этот код обеспечивает случайное распределение данных, чтобы каждый набор (тренировочный, валидационный и тестовый) был репрезентативным.

9. Токенизация:

- Используйте специальную библиотеку, например, transformers от Hugging Face, для токенизации текста, что позволит преобразовать слова в числовые значения, необходимые для работы модели.
- Например, создайте токенизатор для модели GPT:

```

from transformers import GPT2Tokenizer
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

```

- Примените токенизацию к каждому набору данных.

2. Архитектура трансформера

Задание: Изучить основные компоненты архитектуры трансформера и объяснить их роль. Подготовить объяснение работы self-attention и многоголового внимания. Трансформер — это архитектура нейронной сети, разработанная для эффективной обработки последовательностей данных, таких как текст. Она представлена в работе *Attention is All You Need* (2017) авторами Vaswani et al., и стала фундаментом для современных больших языковых моделей (LLM), включая GPT, BERT, T5 и другие. Архитектура трансформера заменяет рекуррентные и сверточные сети и широко применяется в задачах обработки естественного языка (NLP), так как способна параллельно обрабатывать последовательности, эффективно учитывать контекст и учитывать долговременные зависимости.

Основные компоненты архитектуры трансформера

Трансформер состоит из следующих основных блоков:

1. Self-Attention (механизм самовнимания) — определяет взаимосвязи между словами в предложении, учитывая важность каждого слова для других слов.
2. Многоголовое внимание (Multi-Head Attention) — позволяет модели обучаться различным аспектам взаимосвязей в данных.
3. Feed-Forward нейронная сеть — линейные слои, применяемые для каждого слова независимо, улучшающие обучение модели.
4. Слои нормализации и механизм остаточных связей — стабилизируют обучение и помогают предотвратить исчезновение градиента.

Задание: Изучение компонентов трансформера

Для понимания трансформера студенты должны изучить и объяснить работу Self-Attention, многоголового внимания и роль каждого из этих компонентов в архитектуре.

Подробное объяснение и примеры

1. Модуль Self-Attention

Механизм Self-Attention — это ключевая идея трансформера, который позволяет модели учитывать, насколько важным является каждое слово в предложении по отношению к остальным. Например, при анализе предложения "The cat sat on the mat", модель может сосредоточиться на том, как слова "cat" и "mat" связаны друг с другом.

Этапы работы Self-Attention:

- Вычисление матриц Query (Q), Key (K) и Value (V): каждое слово представлено в виде трех разных векторов.
- Расчет Attention Scores: значения взаимосвязей между словами вычисляются по формуле: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ где QK^T используются для определения значимости слов относительно друг друга, а V — для получения итогового представления.

Пример на с использованием numpy:

```
import numpy as np

# Инициализация случайных значений для Query, Key, Value
Q = np.random.rand(4, 64) # Матрица запросов (Query)
K = np.random.rand(4, 64) # Матрица ключей (Key)
V = np.random.rand(4, 64) # Матрица значений (Value)
d_k = K.shape[1]

# Вычисление весов внимания
attention_weights = np.dot(Q, K.T) / np.sqrt(d_k)
attention_weights = np.exp(attention_weights)
attention_weights = attention_weights /
attention_weights.sum(axis=1, keepdims=True)
```

```
# Вычисление выхода attention
attention_output = np.dot(attention_weights, V)
print("Результат self-attention:", attention_output)
```

Этот код показывает, как каждый элемент текста (слово) взаимодействует с другими элементами.

2. Многоголовое внимание (Multi-Head Attention)

Многоголовое внимание позволяет модели захватывать различные аспекты взаимосвязей между словами. Каждый "головной" (head) слой self-attention обрабатывает данные параллельно, изучая разные зависимости между словами.

Принцип работы:

- Модель делит представления Query, Key и Value на несколько "голов".
- Каждая голова выполняет механизм self-attention отдельно.
- Результаты всех голов объединяются и проходят через линейный слой.

Пример кода на для многоголового внимания с использованием PyTorch:

```
import torch
from torch.nn import MultiheadAttention

# Инициализация данных
embed_dim = 64 # Размерность вложения
num_heads = 8 # Количество голов
multihead_attn = MultiheadAttention(embed_dim, num_heads)

# Случайные входные данные
query = torch.rand(10, 3, embed_dim) #
# Последовательность из 10 слов
key = torch.rand(10, 3, embed_dim)
value = torch.rand(10, 3, embed_dim)

# Вычисление многоголового внимания
attn_output, attn_output_weights = multihead_attn(query,
key, value)
print("Результат многоголового внимания:", attn_output)
```

3. Применение self-attention в трансформере

Каждый слой трансформера включает в себя модуль self-attention, многоголовое внимание и feed-forward сеть. Self-attention обрабатывает каждое слово относительно других слов в предложении, создавая контекстуальные представления. Эти представления далее обрабатываются feed-forward сетью, что улучшает модель.

Роль self-attention и многоголового внимания:

- Позволяют модели "видеть" весь контекст предложения.
- Поддерживают параллельную обработку данных, ускоряя обучение и улучшая производительность.

Feed-Forward сети и нормализация

Каждое представление после self-attention проходит через feed-forward сеть, состоящую из линейных слоев, которые применяются к каждому слову.

Механизмы нормализации и остаточных связей помогают предотвратить исчезновение градиентов и стабилизируют процесс обучения.

Основные виды трансформеров и их применение

1. Encoder-Only Трансформеры (например, BERT)
 - Используются для задач классификации, анализа текста, извлечения информации.
 - Структура: только encoder-слои.
2. Decoder-Only Трансформеры (например, GPT)
 - Применяются для генерации текста, автозавершения.
 - Структура: только decoder-слои.
3. Encoder-Decoder Трансформеры (например, T5, BART)
 - Используются для задач, где необходимы и понимание, и генерация текста, например, машинный перевод.
 - Структура: оба компонента (encoder и decoder).

Применение трансформеров и основные случаи использования

1. Обработка естественного языка (NLP): классификация текста, анализ тональности, чат-боты, ответы на вопросы, машинный перевод.
2. Генерация текста: автозавершение текста, генерация ответов, создания диалоговых систем.
3. Извлечение информации: извлечение ключевых слов, классификация и категоризация текста.
4. Мультимодальные модели: применение трансформеров для обработки изображений и текста в единой модели.

Заключение

Архитектура трансформера представляет собой мощный инструмент для работы с большими объемами текстовых данных, позволяя моделям обучаться на последовательностях и учитывать долгосрочные зависимости. Трансформеры широко применяются в NLP и играют ключевую роль в создании современных языковых моделей, таких как GPT и BERT.

1. Настройка и обучение LLM

Задание: Настроить заранее обученную модель GPT, определить гиперпараметры, и обучить модель на подготовленных данных.

Шаг 4.1: Инициализация токенизатора

Используйте библиотеку transformers от Hugging Face для токенизации. Например, для модели GPT-2 мы инициализируем токенизатор с предобученной модели.

```
from transformers import GPT2Tokenizer
```

```
# Инициализация токенизатора для GPT-2
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
```

Шаг 4.2: Токенизация данных

Преобразуйте каждый текст из тренировочного, валидационного и тестового наборов в последовательности токенов.

Используйте метод `tokenizer.encode` или `tokenizer.batch_encode_plus` для обработки сразу нескольких текстов.

```
# Пример токенизации каждого набора данных
train_tokens = [tokenizer.encode(text,
return_tensors='pt', truncation=True) for text in
train_data]
val_tokens = [tokenizer.encode(text, return_tensors='pt',
truncation=True) for text in val_data]
test_tokens = [tokenizer.encode(text,
return_tensors='pt', truncation=True) for text in
test_data]
```

5. Настройка и обучение модели

Шаг 5.1: Загрузка предобученной модели

Используем предобученную модель GPT-2, которая будет настраиваться (fine-tuning) на нашем корпусе данных.

```
from transformers import GPT2LMHeadModel

# Инициализация предобученной модели
model = GPT2LMHeadModel.from_pretrained('gpt2')
```

Шаг 5.2: Настройка гиперпараметров

Определите гиперпараметры, такие как размер мини-батча, число эпох и скорость обучения.

```
from transformers import AdamW

# Настройка гиперпараметров
batch_size = 8
num_epochs = 3
learning_rate = 5e-5

# Инициализация оптимизатора
optimizer = AdamW(model.parameters(), lr=learning_rate)
```

Шаг 5.3: Создание DataLoader

Преобразуйте данные в `DataLoader`, чтобы их можно было обрабатывать партиями. Это необходимо для эффективного обучения модели.

```
from torch.utils.data import DataLoader, TensorDataset

# Пример для тренировочного набора
```

```
train_dataset = TensorDataset(torch.cat(train_tokens,
dim=0))
train_loader = DataLoader(train_dataset,
batch_size=batch_size, shuffle=True)
```

6. Обучение модели

Шаг 6.1: Цикл обучения

Запустите цикл обучения, где на каждом шаге вычисляется функция потерь и обновляются веса модели.

```
import torch

# Основной цикл обучения
model.train()
for epoch in range(num_epochs):
    for batch in train_loader:
        inputs = batch[0].to(device)
        outputs = model(inputs, labels=inputs)
        loss = outputs.loss
        loss.backward()

        optimizer.step()
        optimizer.zero_grad()
    print(f"Эпоха {epoch+1}/{num_epochs}, Потеря:
{loss.item()}")
```

7. Оценка модели

Шаг 7.1: Метрика Perplexity

Вычислите perplexity на валидационном наборе, чтобы оценить, насколько хорошо модель предсказывает текст.

```
import math

model.eval()
total_loss = 0
for batch in val_loader:
    inputs = batch[0].to(device)
    with torch.no_grad():
        outputs = model(inputs, labels=inputs)
        loss = outputs.loss
        total_loss += loss.item()

avg_loss = total_loss / len(val_loader)
perplexity = math.exp(avg_loss)
print("Perplexity:", perplexity)
```

8. Оптимизация модели и сравнение оптимизаторов

Задание: Исследовать влияние различных оптимизаторов и методов управления скоростью обучения на качество обучения модели. Провести эксперименты с

разными оптимизаторами и сделать выводы о том, какой из них лучше подходит для вашей задачи.

Теоретическая часть: Обзор оптимизаторов и scheduler'ов

1. Оптимизаторы

Оптимизаторы играют ключевую роль в процессе обучения нейронной сети, определяя, как обновляются веса модели на каждом шаге. Основные оптимизаторы, которые мы рассмотрим:

- **Stochastic Gradient Descent (SGD)**
- **Adam (Adaptive Moment Estimation)**
- **RMSProp**

SGD (Стохастический градиентный спуск)

- **Описание:** Обновляет веса в направлении отрицательного градиента функции потерь. Стохастичность достигается за счёт использования случайных мини-батчей.
- **Формула обновления весов:** $w = w - \eta \cdot \nabla L(w)$ где η — скорость обучения, $\nabla L(w)$ — градиент функции потерь по весам.

Adam

- **Описание:** Комбинирует идеи AdaGrad и RMSProp. Использует адаптивные скорости обучения для каждого параметра, учитывая моменты первого и второго порядка (среднее и дисперсию градиентов).
- **Преимущества:** Быстрая сходимость, хорошо работает на больших датасетах и с разреженными градиентами.
- **Формула обновления весов:**
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(w_t)$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(w_t))^2$$
$$m_t^{\wedge} = m_{t-1} m_t^{\wedge} + (1 - \beta_1) m_t$$
$$v_t^{\wedge} = v_{t-1} v_t^{\wedge} + (1 - \beta_2) v_t$$
$$w_{t+1} = w_t - \eta \frac{m_t^{\wedge}}{v_t^{\wedge} + \epsilon}$$
 где β_1 и β_2 — коэффициенты экспоненциального затухания, ϵ — малое число для избежания деления на ноль.

RMSProp

- **Описание:** Модификация SGD, которая использует адаптивную скорость обучения, нормализуя градиенты на основе скользящего среднего квадратов предыдущих градиентов.
- **Формула обновления весов:**
$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma) (\nabla L(w_t))^2$$
$$w_{t+1} = w_t - \eta \frac{E[g]_t}{\sqrt{E[g^2]_t + \epsilon}} \nabla L(w_t)$$
 где γ — коэффициент затухания.

2. Learning Rate Scheduler (Планировщики скорости обучения)

Scheduler'ы управляют скоростью обучения во время обучения модели.

Изменение скорости обучения может помочь модели выйти из локальных минимумов и улучшить сходимость.

- **Fixed learning rate:** Постоянная скорость обучения на протяжении всего обучения.
- **Step Decay:** Скорость обучения уменьшается на фиксированный фактор через определённое количество эпох.
- **Exponential Decay:** Скорость обучения экспоненциально уменьшается.
- **Cosine Annealing:** Скорость обучения плавно уменьшается по косинусоидальной кривой.

- **Linear Warmup:** Скорость обучения постепенно увеличивается в начале обучения до заданного значения.

Практическая часть: Эксперименты с оптимизаторами и scheduler'ами

Цель: Провести обучение модели с различными оптимизаторами и scheduler'ами, сравнить результаты и выбрать наиболее эффективную комбинацию.

Шаги решения:

1. **Выбор оптимизаторов и scheduler'ов для экспериментов**
 - Оптимизаторы: SGD, Adam, RMSProp.
 - Scheduler'ы: Без scheduler'a (фиксированная скорость обучения), Linear Warmup с последующим Linear Decay.
2. **Настройка гиперпараметров**
 - Скорость обучения (η): Начните с $5e-5$ для Adam и $1e-3$ для SGD.
 - Другие параметры оставьте по умолчанию или настройте в соответствии с документацией.
3. **Подготовка цикла обучения**
 - Создайте функцию обучения, которая принимает оптимизатор и scheduler в качестве параметров.
 - В этой функции реализуйте цикл обучения с обновлением весов и применением scheduler'a.
4. **Обучение модели с различными оптимизаторами**
 - Обучите модель с каждым оптимизатором отдельно, сохраняя метрики после каждой эпохи (например, потерю и Perplexity на валидационном наборе).
 - Для каждого оптимизатора проведите обучение со scheduler'ом и без него.
5. **Сбор и анализ результатов**
 - Постройте графики потерь и Perplexity для каждого эксперимента.
 - Сравните скорость сходимости, финальные значения потерь и Perplexity.
 - Проанализируйте, как использование scheduler'a влияет на обучение.
6. **Выводы**
 - На основании полученных результатов сделайте выводы о том, какой оптимизатор и scheduler наиболее эффективны для вашей задачи.
 - Обсудите возможные причины различий в производительности.

Реализация

Шаг 1: Импорт необходимых библиотек

```
import torch
from torch.utils.data import DataLoader, TensorDataset
from transformers import GPT2LMHeadModel, GPT2Tokenizer,
AdamW, get_linear_schedule_with_warmup
import torch.optim as optim
import matplotlib.pyplot as plt
```



```
import math
```

Шаг 2: Подготовка данных

У нас данные уже подготовлены, токенизированы и загружены в DataLoader

Шаг 3: Определение функции обучения

```
def train_model(model, optimizer, scheduler, num_epochs,
train_loader, val_loader, device):
    train_losses = []
    val_losses = []
    val_perplexities = []

    model.to(device)

    for epoch in range(num_epochs):
        model.train()
        total_train_loss = 0

        for batch in train_loader:
            inputs = batch[0].to(device)
            model.zero_grad()

            outputs = model(inputs, labels=inputs)
            loss = outputs.loss
            loss.backward()

            optimizer.step()
            if scheduler:
                scheduler.step()

            total_train_loss += loss.item()

        avg_train_loss = total_train_loss /
len(train_loader)
        train_losses.append(avg_train_loss)

        # Оценка на валидационном наборе
        model.eval()
        total_val_loss = 0

        with torch.no_grad():
            for batch in val_loader:
                inputs = batch[0].to(device)
                outputs = model(inputs, labels=inputs)
                loss = outputs.loss
                total_val_loss += loss.item()

        avg_val_loss = total_val_loss / len(val_loader)
        val_losses.append(avg_val_loss)
```

```

        perplexity = math.exp(avg_val_loss)
        val_perplexities.append(perplexity)

        print(f"Эпоха {epoch+1}/{num_epochs}")
        print(f"Средняя потеря на тренировке:
{avg_train_loss}")
        print(f"Средняя потеря на валидации:
{avg_val_loss}")
        print(f"Perplexity на валидации: {perplexity}")
        print("-" * 30)

    return train_losses, val_losses, val_perplexities

```

Шаг 4: Проведение экспериментов

Эксперимент 1: Optimizer - Adam, без scheduler'a

```

device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
num_epochs = 3

# Инициализация модели и оптимизатора
model = GPT2LMHeadModel.from_pretrained('gpt2')
optimizer = AdamW(model.parameters(), lr=5e-5)
scheduler = None # Без scheduler'a

# Обучение модели
train_losses_adam, val_losses_adam, val_ppl_adam =
train_model(model, optimizer, scheduler, num_epochs,
train_loader, val_loader, device)

```

Эксперимент 2: Optimizer - Adam, с scheduler'ом

```

# Инициализация модели и оптимизатора
model = GPT2LMHeadModel.from_pretrained('gpt2')
optimizer = AdamW(model.parameters(), lr=5e-5)

# Инициализация scheduler'a
total_steps = len(train_loader) * num_epochs
scheduler = get_linear_schedule_with_warmup(optimizer,
num_warmup_steps=100, num_training_steps=total_steps)

# Обучение модели
train_losses_adam_sched, val_losses_adam_sched,
val_ppl_adam_sched = train_model(model, optimizer,
scheduler, num_epochs, train_loader, val_loader, device)

```

Эксперимент 3: Optimizer - SGD, без scheduler'a

```

# Инициализация модели и оптимизатора
model = GPT2LMHeadModel.from_pretrained('gpt2')

```

```
optimizer = optim.SGD(model.parameters(), lr=1e-3)
scheduler = None # Без scheduler'a

# Обучение модели
train_losses_sgd, val_losses_sgd, val_ppl_sgd =
train_model(model, optimizer, scheduler, num_epochs,
train_loader, val_loader, device)
```

Эксперимент 4: Optimizer - RMSProp, без scheduler'a

```
# Инициализация модели и оптимизатора
model = GPT2LMHeadModel.from_pretrained('gpt2')
optimizer = optim.RMSprop(model.parameters(), lr=1e-4)
scheduler = None # Без scheduler'a

# Обучение модели
train_losses_rmsprop, val_losses_rmsprop, val_ppl_rmsprop
= train_model(model, optimizer, scheduler, num_epochs,
train_loader, val_loader, device)
```

Примечание: Вы можете добавить больше экспериментов, комбинируя разные оптимизаторы и scheduler'ы.

Шаг 5: Сбор и визуализация результатов

```
# Построение графиков потерь
epochs = range(1, num_epochs + 1)

plt.figure(figsize=(12, 6))
plt.plot(epochs, val_ppl_adam, label='Adam')
plt.plot(epochs, val_ppl_adam_sched, label='Adam +
Scheduler')
plt.plot(epochs, val_ppl_sgd, label='SGD')
plt.plot(epochs, val_ppl_rmsprop, label='RMSProp')
plt.xlabel('Эпоха')
plt.ylabel('Perplexity на валидации')
plt.title('Сравнение Perplexity при разных
оптимизаторах')
plt.legend()
plt.show()
```

Шаг 6: Анализ результатов

- Сравните графики Perplexity для каждого оптимизатора.
- Обратите внимание на скорость сходимости и финальные значения Perplexity.
- Обсудите, как наличие или отсутствие scheduler'a повлияло на обучение.

Выводы

После проведения экспериментов и анализа результатов студенты должны сделать выводы:

- **Какой оптимизатор показал лучшую производительность?**

- Например, если Adam с scheduler'ом показал наименьшую Perplexity и быструю сходимость, можно сделать вывод, что эта комбинация оптимальна для данной задачи.
- **Как использование scheduler'a повлияло на обучение?**
 - Если использование scheduler'a улучшило сходимость или снизило переобучение, это стоит отметить.
- **Возможные причины различий в производительности**
 - Adam и RMSProp адаптивно изменяют скорость обучения для каждого параметра, что может способствовать более эффективному обучению.
 - SGD с фиксированной скоростью обучения может сходиться медленнее и быть чувствительным к выбору скорости обучения.

Заключение

Этот эксперимент демонстрирует важность выбора правильного оптимизатора и стратегии управления скоростью обучения при обучении больших языковых моделей. Студенты не только научатся применять различные оптимизаторы, но и поймут, как анализировать их влияние на процесс обучения и качество модели.

5. Методы оценки качества

1. Метрика BLEU (Bilingual Evaluation Understudy)

Описание: BLEU (Bilingual Evaluation Understudy) — это одна из первых и самых популярных метрик для оценки качества сгенерированного текста, обычно используемая в задачах машинного перевода. Она измеряет, насколько близко сгенерированный текст (предсказание модели) соответствует одному или нескольким эталонным текстам (референс).

Как работает BLEU:

- BLEU вычисляется как среднее точное совпадение (n-граммы) между сгенерированным и эталонным текстом.
- n-граммы — это последовательности из n слов. Например, для биграмм (n=2) последовательность «the cat» является одной биграммой.
- BLEU использует модифицированную точность (precision), которая наказывает за избыточные повторяющиеся n-граммы в предсказании, чтобы избежать «накручивания» оценки.

Формула BLEU:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

где BP — это коэффициент штрафа за длину (brevity penalty), p_n — точность для n-грамм, а w_n — вес для n-грамм.

- Обычно BLEU оценивается на униграммах, биграмах, триграммах и четырехграммах (N=4).
- Brevity Penalty (BP) — это штраф, который применяется, если сгенерированный текст короче эталонного.

Интерпретация:

- BLEU-оценка варьируется от 0 до 1 (или от 0 до 100%), где 1 (100%) указывает на полное совпадение с эталоном.
- BLEU, близкий к 1, говорит о высокой схожести предсказания с эталоном, однако модель может генерировать «слишком точные» тексты, теряя креативность.

Пример использования:

```
from nltk.translate.bleu_score import sentence_bleu

reference = [['this', 'is', 'a', 'test']]
candidate = ['this', 'is', 'a', 'test']
score = sentence_bleu(reference, candidate)
print("BLEU Score:", score)
```

2. Метрика ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Описание: ROUGE — это семейство метрик, разработанных для оценки качества суммаризации текста, но также применяется и для других задач генерации текста. Наиболее часто используются вариации ROUGE-1, ROUGE-2 и ROUGE-L.

Как работает ROUGE:

- ROUGE-1 и ROUGE-2 измеряют точное совпадение униграмм и биграмм между сгенерированным текстом и эталоном. Это похоже на BLEU, но здесь учитывается и точность (precision), и полнота (recall).
- ROUGE-L измеряет наибольшую общую подпоследовательность (Longest Common Subsequence, LCS) между сгенерированным и эталонным текстом, чтобы учесть порядок слов.

Основные формулы:

- ROUGE-1, ROUGE-2 (для униграмм и биграмм):

$$ROUGE - N = \frac{\sum_{S \in \text{References}} \sum_{n\text{-gram} \in S} \min(n\text{-gram}_{\text{generated}}, n\text{-gram}_{\text{reference}})}{\sum_{S \in \text{References}} \sum_{n\text{-gram} \in S} n\text{-gram}_{\text{reference}}}$$

- ROUGE-L:

$$ROUGE - L = \frac{LCS(\text{generated}, \text{reference})}{\text{длина эталона}}$$

Интерпретация:

- ROUGE показывает, насколько хорошо сгенерированный текст покрывает ключевые слова и фразы из эталона.
- Высокий показатель ROUGE говорит о высокой степени совпадения, но, как и BLEU, может не всегда отображать разнообразие и осмысленность текста.

Пример использования:

```
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)
```

```
scores = scorer.score('this is the generated summary',
    'this is the reference summary')
print("ROUGE Score:", scores)
```

3. Метрика Perplexity

Описание: Perplexity — это метрика, которая измеряет степень «неуверенности» модели относительно предсказаний следующего слова в тексте. Она широко используется для оценки языковых моделей. Чем ниже Perplexity, тем лучше модель предсказывает текст, следовательно, тем выше её точность.

Как работает Perplexity:

- Перплексия основана на вычислении вероятности предсказания слов. Чем более вероятно слово, тем ниже Perplexity.
- Она рассчитывается как средняя логарифмическая вероятность последовательности токенов:

$$\text{Perplexity} = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i|w_1, \dots, w_{i-1})}$$

- где N — количество слов, а $P(w_i|w_1, \dots, w_{i-1})$ — вероятность каждого слова, предсказанная моделью.

Интерпретация:

- Perplexity > 1: модель предсказывает слова неуверенно, и чем выше значение, тем хуже её предсказания.
- Perplexity ~ 1: модель хорошо предсказывает вероятности и уверенно «понимает» последовательность.

Пример использования:

```
import math

# Пример расчета Perplexity на основе значений потерь модели
def calculate_perplexity(loss):
    return math.exp(loss)

loss = 2.0 # Пример средней потери
perplexity = calculate_perplexity(loss)
print("Perplexity:", perplexity)
```

Сравнение и использование метрик BLEU, ROUGE и Perplexity

- BLEU и ROUGE используются для оценки совпадения с эталоном в задачах генерации текста и перевода. BLEU лучше подходит для точного совпадения n-грамм, тогда как ROUGE подходит для задач суммаризации, так как учитывает полноту (recall).
- Perplexity эффективна для оценки языковых моделей, так как показывает, насколько модель уверена в своих предсказаниях следующего слова. Обычно используется для измерения качества и устойчивости модели.

Применение на практике

1. BLEU — оценка качества перевода или точного воспроизведения текста.
2. ROUGE — оценка качества суммаризации, обобщения текста.

3. Perplexity — оценка языковых моделей и генерации текста (показатель вероятности текста, сгенерированного моделью).

Эти метрики позволяют объективно оценить качество и точность предсказаний модели и помогают разработчикам выбирать более качественные модели и методы улучшения их работы.

Задание: Оценить модель с помощью BLEU, ROUGE, Perplexity и провести ручную проверку ответов.

Шаги решения

1. Расчет Perplexity

- Perplexity указывает на сложность модели. Чем ниже значение, тем лучше.
- Пример вычисления Perplexity:

```
import math
perplexity = math.exp(loss)
```

2. Метрики BLEU и ROUGE:

- Используйте BLEU для оценки точности, ROUGE для сравнения с эталонными ответами.
- Пример использования библиотеки nltk для BLEU:

```
from nltk.translate.bleu_score import sentence_bleu
reference = [['это', 'пример', 'предложения']]
candidate = ['это', 'пример']
score = sentence_bleu(reference, candidate)
```

3. Ручная проверка:

- Оцените ответы модели для проверки осмысленности и релевантности.

6. Интерфейс для взаимодействия с ботом

Задание: Разработать простой интерфейс для тестирования бота.

Шаги для создания интерактивного чат-бота

1. Загрузка и настройка модели и токенизатора

Для начала убедитесь, что модель и токенизатор уже инициализированы и готовы к использованию. Если модель еще не загружена, можно использовать следующую команду:

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Загрузка модели и токенизатора
model = GPT2LMHeadModel.from_pretrained('gpt2')
```

```
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
```

Модель GPT-2 выбрана как базовый пример, но можно использовать любую другую предобученную модель из библиотеки transformers.

2. Основной цикл взаимодействия с пользователем

Теперь создадим цикл, который будет принимать ввод пользователя, обрабатывать его и выводить ответ от модели. Для этого используем команду `input()` для ввода текста и `print()` для вывода ответа.

```
while True:
    input_text = input("Введите ваш запрос: ")
    if input_text.lower() in ["exit", "quit"]:
        print("Чат завершен. До свидания!")
        break

    # Токенизация ввода
    inputs = tokenizer.encode(input_text,
return_tensors='pt')

    # Генерация ответа
    outputs = model.generate(inputs, max_length=50,
do_sample=True)

    # Декодирование и вывод ответа
    response = tokenizer.decode(outputs[0],
skip_special_tokens=True)
    print("Ответ бота:", response)
```

Пояснение к коду:

- `input("Введите ваш запрос: ")` — принимает текст от пользователя.
- `if input_text.lower() in ["exit", "quit"]` — позволяет завершить чат при вводе команд "exit" или "quit".
- `tokenizer.encode()` — преобразует текст пользователя в числовые представления (токены), которые может обработать модель.
- `model.generate()` — генерирует ответ на основе токенов ввода. Параметр `max_length=50` ограничивает длину ответа, а `do_sample=True` добавляет элемент случайности для более естественных ответов.
- `tokenizer.decode()` — преобразует сгенерированные токены обратно в текст.

3. Настройка параметров генерации

Параметры генерации можно настроить, чтобы управлять характером ответов модели.

- `max_length`: максимальная длина ответа.
Например, `max_length=100` позволит получить более длинный ответ.
- `do_sample`: включает случайность при выборе слов. Если установить `do_sample=False`, модель будет генерировать более предсказуемые ответы.

- top_k и top_p: параметры для управления вероятностью выбора слов, что позволяет добиться баланса между творческими и осмысленными ответами.

Пример настройки параметров:

```
outputs = model.generate(
    inputs,
    max_length=50,
    do_sample=True,
    top_k=50,          # Считает только 50 самых вероятных
                        # вариантов
    top_p=0.95         # Выбирает слова с общей вероятностью
                        # не более 0.95
)
```

4. Обработка ошибок и исключений

Чтобы чат-бот работал стабильно, добавьте обработку ошибок на случай, если что-то пойдет не так.

```
while True:
    try:
        input_text = input("Введите ваш запрос: ")
        if input_text.lower() in ["exit", "quit"]:
            print("Чат завершен. До свидания!")
            break

        inputs = tokenizer.encode(input_text,
            return_tensors='pt')
        outputs = model.generate(inputs, max_length=50,
            do_sample=True)
        response = tokenizer.decode(outputs[0],
            skip_special_tokens=True)
        print("Ответ бота:", response)

    except Exception as e:
        print("Произошла ошибка:", str(e))
        print("Попробуйте еще раз.")
```

5. Завершение работы чата

Чтобы завершить чат, пользователь может ввести команду "exit" или "quit". Код проверяет ввод на наличие этих команд и завершает цикл.

```
if input_text.lower() in ["exit", "quit"]:
    print("Чат завершен. До свидания!")
    break
```

Итоговый код для интерактивного чат-бота
Вот итоговый код, объединяющий все шаги:

```

from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Загрузка модели и токенизатора
model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

print("Добро пожаловать в чат! Введите ваш запрос или  
'exit' для выхода.")

# Основной цикл чата
while True:
    try:
        input_text = input("Введите ваш запрос: ")

        # Завершение работы
        if input_text.lower() in ["exit", "quit"]:
            print("Чат завершен. До свидания!")
            break

        # Токенизация и генерация ответа
        inputs = tokenizer.encode(input_text,
return_tensors='pt')
        outputs = model.generate(
            inputs,
            max_length=50,
            do_sample=True,
            top_k=50,          # Учитываются 50 наиболее
вероятных токенов
            top_p=0.95        # Выбор слов с общей
вероятностью не более 0.95
        )

        # Декодирование и вывод ответа
        response = tokenizer.decode(outputs[0],
skip_special_tokens=True)
        print("Ответ бота:", response)

    except Exception as e:
        print("Произошла ошибка:", str(e))
        print("Попробуйте еще раз.")

```

Дополнительные задания

1. Изменение параметров генерации:
 - Попробуйте изменить max_length, top_k, и top_p и оцените, как меняется стиль и качество ответов бота.
2. Анализ ответов:

- Задайте несколько разных вопросов и оцените осмысленность и качество ответов. Какие темы модель понимает лучше?
3. Добавление логирования:
- Реализуйте логирование всех запросов и ответов в текстовый файл для дальнейшего анализа.
4. Создание графического интерфейса (опционально):
- Создать графический интерфейс (например, с помощью библиотеки `streamlit`), чтобы сделать чат более интерактивным.